

第5章 函数设计与使用

高级话题， 自学， 不作要求

高级话题1 lambda表达式

5.6 lambda表达式

- **lambda表达式**用来定义一个仅仅包含**一行代码的匿名函数**，即没有函数名字，函数体只可以包含一个表达式，且该表达式的计算结果为函数的返回值。
- 不允许包含其他复杂的语句，只能是**表达式**，但是注意函数调用也是表达式，因此可以调用**print、map、list**等
- lambda表达式也是函数的一种，是匿名函数，因此也遵循前面介绍过的函数的参数定义和传递、函数的作用域所使用的规则

```
def __anonymous_function__(arguments):  
    return expr
```



```
lambda arguments: expr
```

```
f=lambda x,y,z:x+y+z  
print (f(1,2,3))  
  
def f(x,y,z):  
    return x+y+z
```

```
g=lambda x,y=2,z=3:x+y+z  
print (g(2,z=4,y=5))  
print (g(2,z=4))
```

```
11  
8
```

含有默认参数，调用时使用关键参数

5.6 lambda表达式

- lambda表达式是匿名函数，避免了可能的名字冲突
- 可以用作Code Jump Table: 代码跳转表
- 可以用于那些参数为函数对象的函数，比如map、reduce、filter、sorted等

```
L = [lambda x: x ** 2,  
      lambda x: x ** 3,  
      lambda x: x ** 4]  
  
for f in L:  
    print(f(2))  
print(L[0](3))
```

多个匿名函数对象保存在一个列表中，可以通过下标或者for循环来获得该函数对象，然后调用该函数

```
key = 'f1'  
  
D = {'f1': (lambda: 2 + 2),  
      'f2': (lambda: 2 * 4),  
      'f3': (lambda: 2 ** 6)}  
  
print(D[key]())
```

多个匿名函数对象保存在一个字典中，可以通过key来获得函数对象，然后调用该函数

5.6 lambda表达式

- lambda表达式其他示例(P140页在Python3有问题, data=range(20)→ data=list(range(20))

```
def test_lambda():  
    data=list(range(20))  
    import random  
    random.shuffle(data)  
    print(data)  
    data.sort(key=lambda x:len(str(x)))  
    print(data)  
    data.sort(key=lambda x:x)  
    print(data)  
    data.sort(key=lambda x:len(str(x)),reverse=True)  
    print(data)
```

```
[15, 10, 7, 8, 6, 11, 2, 16, 9, 17, 4, 13, 18, 1, 5, 3, 19, 14, 12, 0]  
[7, 8, 6, 2, 9, 4, 1, 5, 3, 0, 15, 10, 11, 16, 17, 13, 18, 19, 14, 12]  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]  
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

高级话题2 嵌套函数

嵌套函数

- 在函数体内再定义新的函数
- Python中函数也是对象

```
def linear(a, b):  
    def result(x):  
        return a * x + b  
    return result
```

```
func_1_1 = linear(1, 1)  
func_2_1 = linear(2, 1)  
print('f(x) = x+1, f(10) = ', func_1_1(10))  
print('f(x) = 2x+1, f(10) = ', func_2_1(10))
```

```
>>> print('f(x) = x+1, f(10) = ', func_1_1(10))  
f(x) = x+1, f(10) = 11  
>>> print('f(x) = 2x+1, f(10) = ', func_2_1(10))  
f(x) = 2x+1, f(10) = 21
```

- 自由变量：在函数体内未赋值但是使用的变量
- 在result函数定义时,a和b为自由变量

按照LEGB规则，自由变量继续在外层函数中由里到外寻找。

- 每次调用linear返回一个新的函数对象func,
 - 返回的函数对象不再依赖于外层函数linear（实际上已经返回退出了）
 - 外层函数return时进行非全局的自由变量的绑定
 - 保存在 func.__closure__中

含有自由变量的代码块称为闭包(closure)

嵌套函数的late binding

- 不是在定义函数时绑定，而是在定义函数的函数体中返回时绑定非全局的自由变量
- 在函数调用时绑定全局变量

与global语句类似，如果在内层函数要对于外层的自由变量赋值，需要使用nonlocal语句表示函数体赋值不会变为局部变量，而是外层函数的自由变量

```
def makeActs():  
    funcs = []  
    for i in range(3):  
        j = [i]  
        print(hex(id(j)))  
        def func(n):  
            print(n*j[0])  
            print(hex(id(j)))  
        funcs.append(func)  
    return funcs
```

```
acts = makeActs()  
[act(10) for act in acts]
```

```
0x1148da0  
0x113d698  
0x11478f0  
20  
0x11478f0  
20  
0x11478f0  
20  
0x11478f0
```

期待输出为：

0*n, n, *2n (n=10),即
0,10,20

实际输出：20,20,20

可以看到自由变量j都指向同一个对象

嵌套函数的late binding (补充, 不作要求)

```
def makeActs():  
    funcs = []  
    for i in range(3):  
        j = [i]  
        print(hex(id(j)))  
        def func(n):  
            print(n*j[0])  
            print(hex(id(j)))  
        funcs.append(func)  
    return funcs
```

```
acts = makeActs()  
[act(10) for act in acts]
```



```
def makeActs2():  
    funcs = []  
    for i in range(3):  
        j = [i]  
        print(hex(id(j)))  
        def func(n, j=j):  
            print(n*j[0])  
            print(hex(id(j)))  
        funcs.append(func)  
    return funcs
```

```
acts2 = makeActs2()  
[act(10) for act in acts2]
```

```
0x1133440  
0x113d080  
0x1142378  
0  
0x1133440  
10  
0x113d080  
20  
0x1142378
```

高级话题3 可调用对象

可调用对象

- 函数可以调用，实际上函数通过class function来实现
- 自己定义的类如果实现了__call__()方法，则也是可调用对象

```
def linear(a, b):  
    def result(x):  
        return a * x + b  
    return result
```

```
dir(linear)  
dir(linear2)
```

```
class linear2:  
    def __init__(self, a, b):  
        self.a, self.b = a, b  
    def __call__(self, x):  
        return self.a * x + self.b  
  
taxes=linear2(0.3,2)  
print(taxes(5))    # 3.5
```

高级话题4

修饰 (装饰 decorator) 器

修饰器（装饰器）

- 比如：在函数调用前后自动打印日志，但又不希望修改原先函数的定义，这种在代码运行期间动态增加功能的方式，称之为“**修饰器**”或“**装饰器**”（Decorator）。
- 修饰器（decorator）是函数嵌套定义的另一个重要应用。**修饰器本质上也是一个函数**，只不过这个函数接收其他函数作为参数并对其进行一定的改造之后使用新函数替换原来的函数。
- Python面向对象程序设计中的静态方法、类方法、属性等也都是通过修饰器实现的。

修饰器

```
def before(func):                                #定义修饰器，所以接受一个函数作为参数，并返回一个函数
    def wrapper(*args, **kwargs):
        print('Before function called.')
        return func(*args, **kwargs)
    return wrapper
```

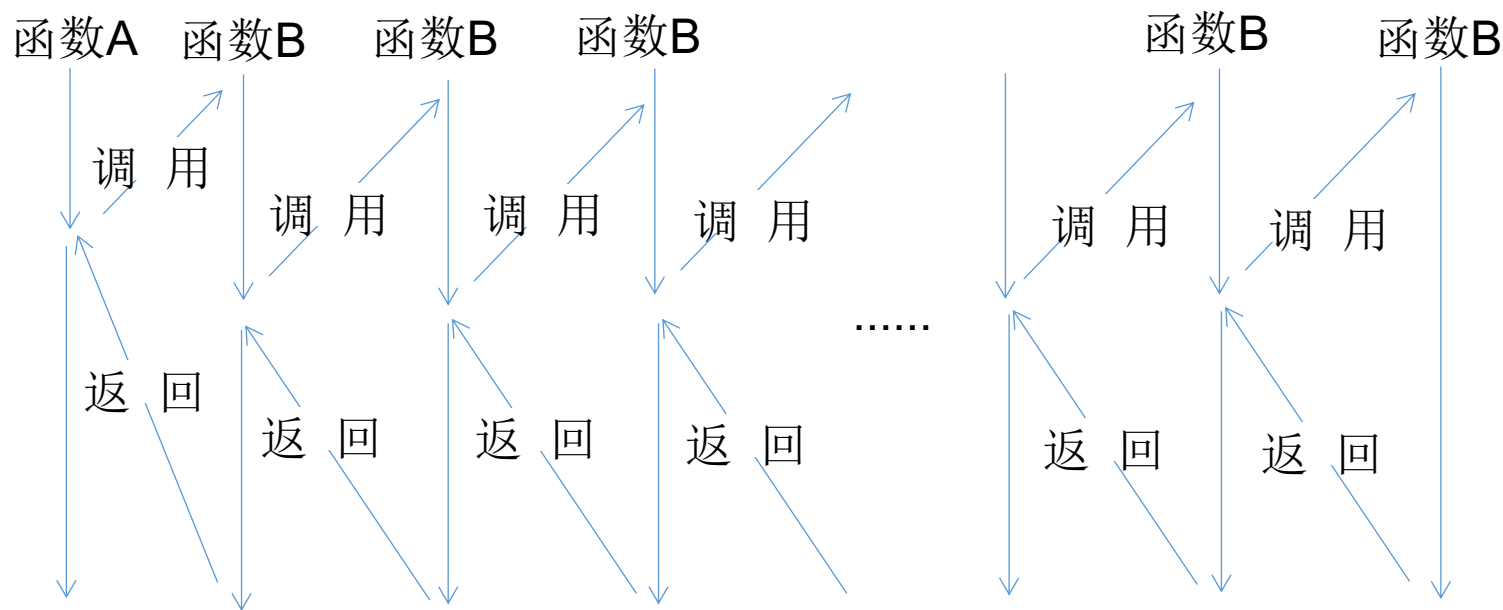
```
def after(func):                                #定义修饰器
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print('After function called.')
        return result
    return wrapper
```

```
@before
@after
def test():                                    # 使用@语法，把decorator置于函数的定义处
    print(3)                                  #同时使用两个修饰器改造函数
#调用被修饰的函数
test()
```

高级话题5 递归函数

函数递归调用

- 函数的**递归调用**是函数调用的一种特殊情况，函数调用自己，自己再调用自己，自己再调用自己，...，当**某个条件得到满足的时候就不再调用了**，然后再**一层一层地返回**直到该函数第一次调用的位置。



递归函数

- 实现起来非常方便，但应注意递归调用不要一直进行下去，没有终止
 - 递归调用的深度是受到限制的
- 效率较低，如果有可能通过循环实现

```
def factorialRecur(n): #阶乘，通过递归实现
    if n == 1:
        return 1
    else:
        return n * factorialRecur(n - 1)
```

```
def factorial(n): #阶乘，通过loop实现
    result = n
    for i in range(1, n):
        result *= i
    return result
```

```
import sys
print(sys.getrecursionlimit()) # 1000
old = sys.getrecursionlimit()
sys.setrecursionlimit(50)
print(factorialRecur(45))
sys.setrecursionlimit(old)
```

"C:\Users\dlmao\PycharmProjects\pythonClass\function.py", line 540, in factorialRecur

if n == 1:
RecursionError: maximum recursion depth exceeded
in comparison

例 · • 使用递归法对整数进行因数分解

```
· from random import randint
```

```
def factors(num, fac=[]):  
    #每次都从2开始查找因数  
    for i in range(2, int(num**0.5)+1):  
        #找到一个因数  
        if num%i == 0:  
            fac.append(i)  
            #对商继续分解, 重复这个过程  
            factors(num//i, fac)  
            #注意, 这个break非常重要  
            break  
    else:  
        #不可分解了, 自身也是个因数  
        fac.append(num)
```

```
facs = []  
n = randint(2, 10**8)  
factors(n, facs)  
result = '*'.join(map(str, facs))  
if n==eval(result):  
    print(n, '= ' + result)
```

高级话题6

map reduce filter

map函数

- 内置函数`map`可以将一个函数作用到一个序列或迭代器对象上。
- `map(func,iterable)`:生成一个支持iterator协议的map对象，其每个元素为对于iterable对象的元素调用相应函数func的结果

```
[str(i) for i in range(5)]
```

```
>>> list(map(str, range(5)))  
['0', '1', '2', '3', '4']  
  
>>> def add5(v):  
        return v+5  
  
>>> list(map(add5, range(10)))  
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

filter

- 内置函数`filter`将一个函数作用到一个序列上，返回该序列中使得该函数返回值为True的那些元素组成的列表、元组或字符串。
- 可以看到map和filter实际上都可以通过以前介绍的list comprehension来实现
- 实际上早期python仅仅支持map/reduce/filter函数，后来才扩展为支持列表解析。

```
>>> seq=['foo', 'x41', '?!', '***']
>>> def func(x):
        return x.isalnum()
>>> list(filter(func, seq))
['foo', 'x41']

>>> seq
['foo', 'x41', '?!', '***']
>>> [x for x in seq if x.isalnum()]
['foo', 'x41']
>>> list(filter(lambda x:x.isalnum(), seq))
['foo', 'x41']
```

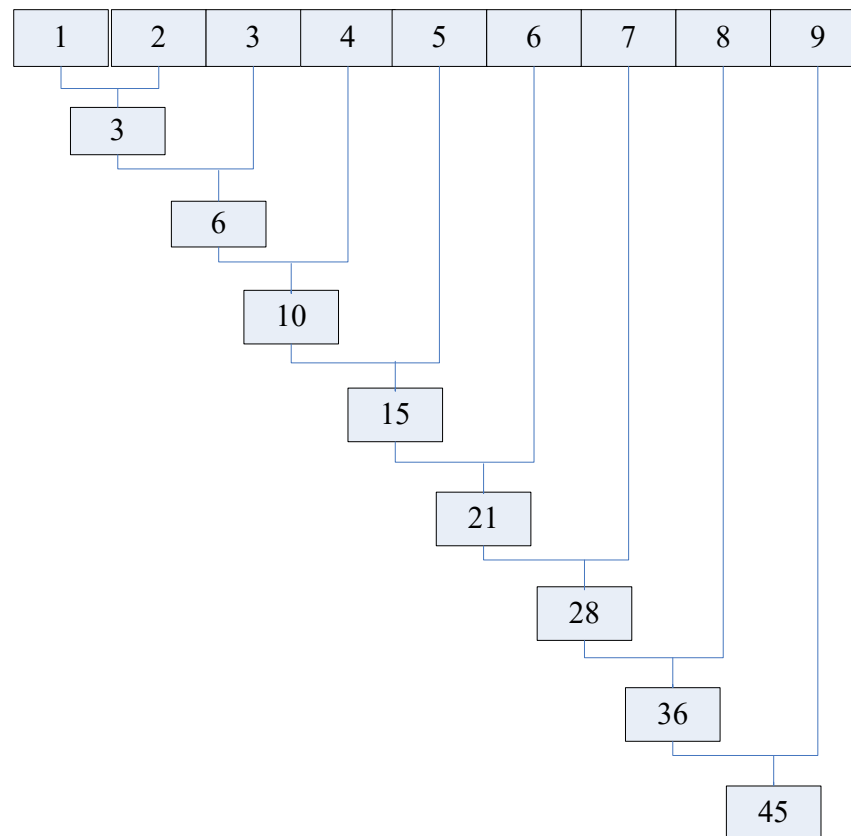
reduce

- ~~内置函数~~reduce可以将一个接受2个参数的函数以累积的方式从左到右依次作用到一个序列或迭代器对象的所有元素上。

reduce(func,iterable): 从第一个元素开始调用func(sum,element), 第一个参数为前面计算的结果, 第二个参数为当前元素

- 在python 3中, reduce函数在模块functools中

```
>>> from functools import reduce
>>> seq=[1,2,3,4,5,6,7,8,9]
>>> reduce(lambda x,y:x+y, seq)
45
>>> def add(x, y):
    return x + y
>>> reduce(add, range(10))
45
>>> reduce(add, map(str, range(10)))
'0123456789'
>>>
```



高级话题7 生成器函数： yield语句

生成器函数：yield语句

- 包含yield语句的函数可以用来创建生成器对象，这样的函数也称生成器函数。
- yield语句与return语句的作用相似，都是用来从函数中返回值。与return语句不同的是，return语句一旦执行会立刻结束函数的运行，而每次执行到yield语句并返回一个值之后会暂停或挂起后面代码的执行，下次通过生成器对象的__next__()方法、内置函数next()、for循环遍历生成器对象元素或其他方式显式“索要”数据时恢复执行。
- 生成器具有惰性求值的特点，适合大数据处理。

例1

```
>>> def f():  
    yield from 'abcdefg'
```

#使用yield表达式创建生成器

```
>>> x = f()
```

```
>>> next(x)
```

```
'a'
```

```
>>> next(x)
```

```
'b'
```

```
>>> for item in x:  
    print(item, end=' ')
```

#输出x中的剩余元素

```
c d e f g
```

例2

```
>>> def gen():  
    yield 1  
    yield 2  
    yield 3
```

```
>>> x, y, z = gen()
```

#生成器对象支持序列解包

例3 斐波那契数列

```
>>> def f():  
    a, b = 1, 1          #序列解包，同时为多个元素赋值  
    while True:  
        yield a          #暂停执行，需要时再产生一个新元素  
        a, b = b, a+b     #序列解包，继续生成新元素  
>>> a = f()              #创建生成器对象  
>>> for i in range(10):   #斐波那契数列中前10个元素  
    print(a.__next__(), end=' ')  
1 1 2 3 5 8 13 21 34 55
```

例3 斐波那契数列

```
>>> for i in f():  
    if i > 100:  
        print(i, end=' ')  
        break
```

#斐波那契数列中第一个大于100的元素

144

```
>>> a = f()  
>>> next(a)
```

元素

1

```
>>> next(a)
```

1

```
>>> a.__next__()
```

2

```
>>> a.__next__()
```

3

#创建生成器对象

#使用内置函数next()获取生成器对象中的

#每次索取新元素时，由yield语句生成

#也可以调用生成器对象的__next__()方法

例4 使用生成器模拟了标准库itertools中的count()函数

```
>>> def count(start, step):  
    num = start  
    while True:                                #无穷循环  
        yield num                               #返回一个数, 暂停执行, 等待下一次索要数据  
        num += step  
>>> x = count(3, 5)  
>>> for i in range(10):  
    print(next(x), end=' ')  
3 8 13 18 23 28 33 38 43 48  
>>> for i in range(10):  
    print(next(x), end=' ')  
53 58 63 68 73 78 83 88 93 98
```